LLossless: catching what a language model silently drops, changes or invents when it merges documents

Toby Sutor

October 16, 2026
Published set `43fda72e9e49`

Abstract

Teams keep the same knowledge in several documents. Merging them is careful, unrewarding work, and people now hand it to language models. What a model drops, changes or invents in a merge leaves no mark on the page. Finding it means rereading the sources, which is the work the merge was meant to save.

LLossless (the name is LLM plus lossless) merges documents with a model of the user's choice and then checks the result against the sources, in both directions. A program compares the text: every source sentence must be in the merge or accounted for, and no number, link, file path or code may change. A model judges the rest, quoting its evidence, and the tool checks that the quote is really there. A merge that fails a check ends the run as failed, so that a pipeline stops instead of publishing it.

I measured 8 configurations, each a model and the way it is reached, on 9 document pairs. A program flags each source sentence it cannot find in a merge, unless a note explains it. A flag says where to look. An AI model, and no person, read every sentence that a merge changed or left out without a note, and by that reading 3 of the 8 configurations lost content with no note. My first scoring rule took every flag for a loss and counted far more. I amended the rule after the results and publish both scorings. Every configuration came closer to a correct merge written in advance than a script that pastes the documents together. In a second test I planted factual errors and let the model correct what it knows is wrong. There, among the 8 configurations, the most expensive configuration fixed the most errors.

Every measured figure in this paper is generated from a recorded run and checked each time the paper is built. I built the tool by directing AI coding agents. They wrote the code, the tests and this text, and several of the lessons here are about checking their work.

1 A merge that reads well and is wrong

Why this is a common problem. Teams write the same knowledge down more than once. A support team keeps a pair of articles about the same failure: one written for every release, the other for a single customer whose incident looked the same. Nobody folds them together, because that is a careful, unrewarding job. Every sentence in each has to end up somewhere, or be dropped on purpose, by someone who can say why. This is the job people now hand to a language model, and the model is very good at the part a reader can see.

The part a reader cannot see is what was left out. A merge that drops a quarter of one source reads exactly like a merge that drops nothing: shorter, cleaner, no gap where the missing material used to be. Where the sources disagree, the value the model picked reads as decided, not guessed. A detail the model invented is written in the same confident voice as the real ones. None of these

leaves a mark on the page. Finding them means reading the sources beside the output, which is the work the merge was supposed to save.

One real case. A bike-share operator has a runbook for its field crew and another for its support desk. The first sends a jammed dock to the workshop after 2 failed attempts, the second after 3. Asked to merge them, Opus 5.5 returned a runbook that reads well. It says 2, it never mentions 3, and nothing on the page shows that the sources disagreed. The model had not overlooked it. At the default setting it is told to choose one value where the sources disagree and to note the choice in a *record*, a short note about a source sentence that it did not copy word for word. It wrote that record: "Threshold conflict resolved in favour of the base value." A reader of the runbook never sees it.

LLossless caught it (Figure 1). Its report lists the support desk's sentence, at line 15 of its runbook, as contradicted by the merge, and ends the run with exit code 1, which means a problem in the merged document. Which number is right is for a person to decide. Without the report, nobody would know there was a decision to make. Every pair in my tests comes with a *reference merge*, a correct merge written in advance. The one for this pair keeps 2 as well, because the field crew's runbook gives the reason: forcing a jammed dock again bends the pin. So the model chose well, and still nothing on its page tells a reader that a choice was made. Had it chosen 3, the page would have read just as well.

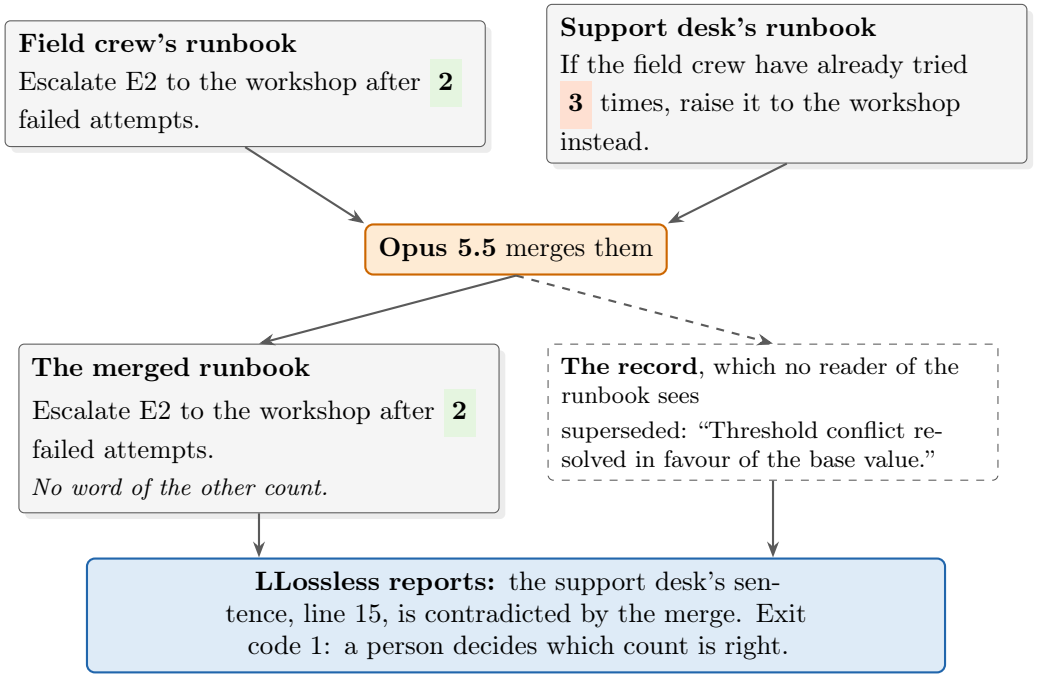


Figure 1: The opening case. The merge chose the field crew's count, which is the right one, and noted the choice in a record that the merged runbook does not show. Had it chosen the other count, the page would have read just as well. The report is what makes the choice visible.

It was not one model's quirk. Of the 8 configurations in my benchmark (a configuration is one model, reached one way), 7 merged the pair the same way. Whether a merge contradicts a source is a question about meaning, so LLossless puts it to a model, which I call the *judge*. In the benchmark the judge was the model that had written the merge, and it reported the conflict on 5 of those 7

merges. On the others the model judging its own merge did not report it. That is why the tool keeps the checks that depend on a judge apart from the checks a program does alone (Section 3).

Declared loss and silent loss. This paper depends on the difference between these, so I state it here. A *declared loss* is content that the merge left out and said so, in a record. That is usually fine, and often what a merge should do. A reviewer can see it, and the tool checks the record. The runbook merge above is one: it left out the support desk's count and said so. A *silent loss* is content that is missing with no record. Finding those is exactly what this tool is for. A program looks for every source sentence in the merge, and a sentence it cannot find and that no record explains is counted as a silent loss. That count is a place to look and not proof, because the program compares text and also flags a sentence that the merge kept in other words. By the count the runbook merge is clean, with 0 silent losses, because its loss was declared. LLossless failed the run all the same, because the judge found that the merge contradicts a source.

A model on guardrails. The use of language models for real work is held back by trust and by cost. A model can state things that are not so, which is called hallucination, and it can quietly leave things out. And every use costs money. LLossless is built with both in mind.

On trust, the tool does not ask the model to be reliable. It puts the model on guardrails. The model does the writing. A program, which cannot hallucinate, checks everything that can be checked by comparing text: that every source sentence is accounted for, and that every number, link, file path and code block is unchanged. A model is asked only where meaning has to be judged. It must quote its evidence, and the tool checks that the quote is there. The run ends with an exit code, so an automated pipeline stops on a bad merge instead of publishing it. The limit should be stated just as plainly. The guardrails do not stop a model from making an error. They make the error visible and stop the run. Even that has limits. A program cannot tell a sentence in other words from a lost one, or a sentence that lost a clause from a reworded one, so a person has to look. And where a judge is needed, the judge can miss (Section 3).

On cost, every report has a line for what the run cost, or for why the cost is not known. Writing the merge and checking it are separate jobs in a run, and each job can go to a different model, so an expensive model can write while a cheaper one checks (Section 2). A cheaper depth of checking exists, and Section 4.6 measures what it gives up. And the benchmark measured what a merge costs on each model. In it the configuration closest to the reference merges cost about a third of the most expensive one. It was also the cheapest of those that the benchmark did not disqualify (Section 4.2).

What the tool does. The guardrails enforce one rule. Everything in the sources is either in the merge or declared as left out, and everything in the merge comes from a source. The tool merges documents with a model of the user's choice and then checks that rule in both directions. The checks that a program does alone, with no model, I call the *mechanical checks*. They find a source sentence that is missing and has no record, and a changed number, link or piece of code. The checks that rest on the judge I call the *judged checks*. They are made one statement at a time, with a quote for every verdict. The result is a report a person can check line by line and an exit code a pipeline can act on.

What I measured. Because the mechanical checks are a program, they can score many models on the same documents in the same way. Section 4 reports one such benchmark, run in a single session that began on 2026-09-27: 8 configurations of hosted models (models that their vendors run, reached over the network), each merging the same 9 document pairs. The program counted a silent loss for 4 of the 8 on at least one pair. A count is only a place to look, so an AI model, and no

person, read each counted sentence against its merge. On the first run of each pair it also read every sentence that the program had passed as reworded although no record names it. For 3 of the 8 the reading found content gone with no record, and the benchmark disqualifies those. One of them is out for a single sentence that lost its opening condition, a case the readings call borderline.

My first scoring rule let the count decide. The reading showed that most of what it had counted was still in the merge, so I amended the rule after the results were known. Both scorings are published, and Section 5.3 tells how the error was found. Neither scoring counts a loss that the merge declared, such as the dropped count in the runbook merge. Section 7 comes back to that.

Contributions.

- A tool that puts a language model on guardrails for one job, merging documents. The model writes the merge and the tool audits it. The mechanical checks ask no model, and they are kept apart from the judged checks, which need one.
- A dated benchmark of current hosted models on merging without loss. It is published under the scoring as registered and under a scoring amended after the results, with the audit by reading that led from the first to the second.

Where the test documents come from. The test documents are a handful of short texts written for this project, by me and by the coding agents, plus excerpts adapted from Wikipedia and from one NASA page. The repository says who wrote which and under what licence.
The terms this paper uses, with the meaning each has here, are listed in Appendix B.

2 Using it

This section shows the tool from the outside: its commands, the one setting that matters most, and what a run returns. How the checks work inside is the subject of Section 3.

A check that nobody runs protects nobody. So LLossless sits where merges already happen: as a command in a script or a pipeline, and as a web page. It has two jobs. `llossless merge` asks a model to merge up to 12 documents and then checks the result. `llossless verify` checks a merge that already exists, however it was made, and makes no merge call.

```
llossless merge a.md b.md --base a.md -o merged.md > report.md
llossless verify a.md b.md merged.md > report.md
```

In the first command `a.md` and `b.md` are the sources. `--base` names the *base*: the source whose structure the merged document follows. `-o` names the file the merged document is written to, and the report goes to standard output. The second command takes the sources and then the merge to be checked.

The web interface. `llossless serve` runs the same merge and the same checks behind a small self-hosted server (Figure 2). It has accounts that can each hold their own API keys, a model picker that shows measured cost, speed and silent loss per model with the date of the measurement and marks a model that the benchmark disqualified, and pages in English and German.

Three roles, and three ways to reach a model. A model does up to three jobs in a run, and I call them roles. In the *merge* role it writes the merged document. In the *decompose* role it splits each document into *claims*, which are short statements of one fact each. In the *verify* role it compares each claim with the other side: a claim from a source with the merged document, and a claim from the merged document with the sources. Decompose and verify together are the judge. The roles can use different models, so an expensive model can write the merge while a cheaper one checks it.

A model can be reached through a local server that speaks the OpenAI API, through a vendor's API, or through a command-line program that runs under a subscription, such as Anthropic's Claude Code. A vendor's API is billed by the token, and a subscription is a fixed fee that buys a budget of usage (Section 4 comes back to the difference). The tool starts Claude Code in its safe mode, an option that loads none of the user's own configuration (Section 6 says why).

How freely the merge may reword. A merge that may only copy sentences is wrong when it rewrites one, and a merge that may edit freely is not. So one setting, the *fidelity level*, tells the merge how much it may edit and tells the checks the same thing. There are 6 levels, each allowing what the rows above it allow and more. The default is `high`, because I expect a model to improve how the merged text reads and flows. `verbatim` and `low` keep the text with little or no change, and I doubt that `verbatim` has much use in practice.

Level	The merge may
`verbatim`	only copy sentences, choosing which version to keep and in what order
`low`	also make mechanical corrections: spelling, punctuation, grammar
`mid`	also rewrite single sentences
`high`	also edit like a careful editor, state what the sources together entail, and choose one value where the sources disagree, with a record of the choice
`open`	also carry a value that covers both sources where they disagree, and state something from outside the documents, in a declared record that names what it rests on
`sourced`	as `open`, with the model looking facts up on the web instead of recalling them

At `high` the merge may join facts from several documents into one statement, and where the sources disagree it may choose one value and record the choice, as the runbook merge did. Up to `mid` it may not choose, so a user who wants every disagreement kept on the page runs at `mid` or below. At `open` the merge may also state something that is in none of the documents, for example to correct an error in a source, which it would otherwise carry over. The tool cannot check such a statement against the documents, so the merge must declare each one with what it rests on, and the report lists them for a person to review.

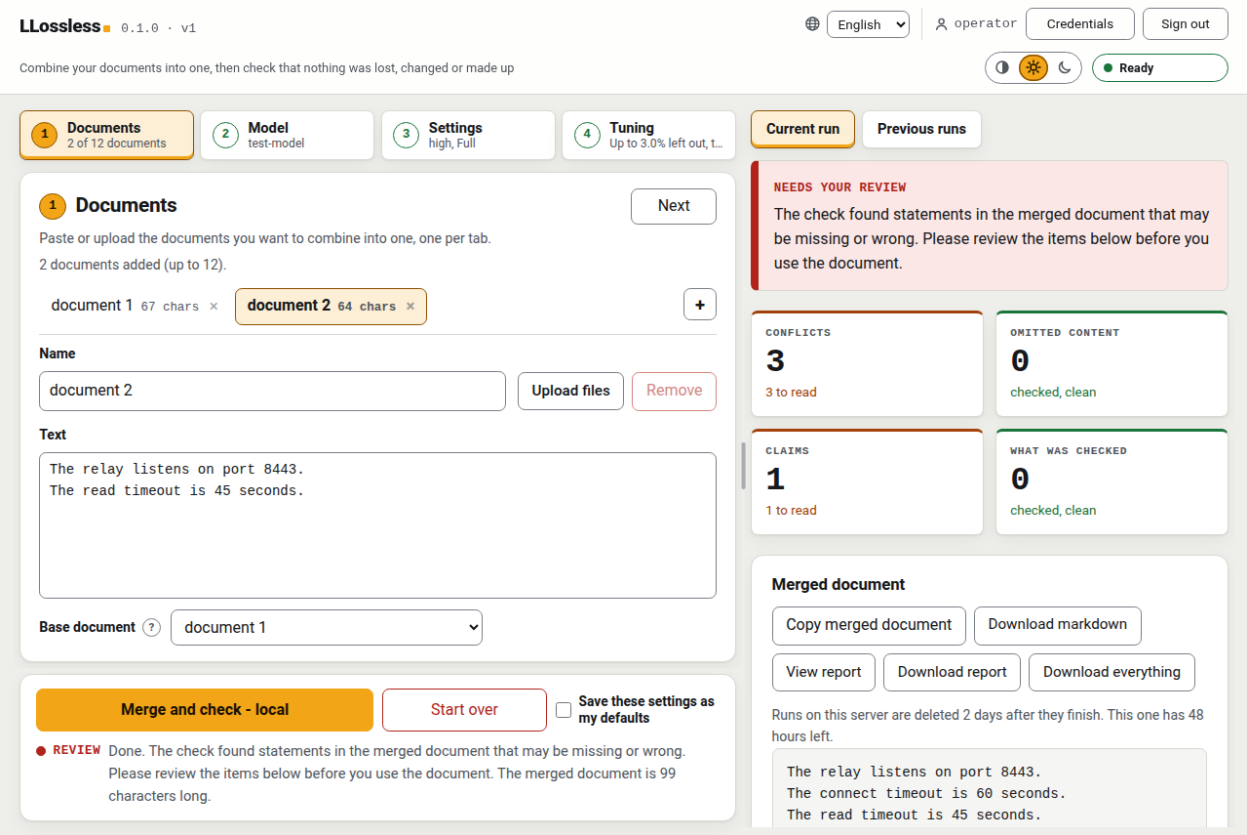


Figure 2: The web interface after a run with findings: the documents on the left, and on the right the findings, the checks that ran and the merged document with its downloads. The answers in this screenshot came from a scripted test endpoint, not from a real model.

Rules that do not depend on the level. In a merge the tool made, numbers, links, file paths, version strings and code must arrive character for character at every level. A changed one is a finding.

What a merge states from outside the documents has three possible outcomes at `open`, and the tool treats them differently:

- *Declared, and the documents say nothing on the matter.* The statement is listed in the report for review. It is not a finding.
- *Declared, and it contradicts a source.* This is a correction. It is listed in the report, and it is still a finding that fails the run. The tool cannot tell a correction from an error, because the documents are the only reference it has.
- *Not declared.* The statement is reported as invented.

One run of the benchmark shows the second case. A source of the Voyager set says that the names of some newly found moons are "obvious allusions to Goethe". That is one of the errors I planted. At `open` the model wrote "allusions to Shakespeare and Alexander Pope" and declared the change. Shakespeare and Alexander Pope is the correct fact and Goethe is the error I planted, so the model put the planted error right. The report still lists the statement among those added from outside the documents. It also reports that the merge contradicts its source, and the run ends with exit

code 1, because the tool cannot tell a correction from a contradiction. A person has to confirm the correction.

What comes back. A run returns a report and an exit code. The report is in Markdown, and on request the same report comes as JSON or as a self-contained HTML page. It opens with a summary of what was found. Then come the findings. A *finding* is an entry that names one problem. Each finding of the judged checks names the claim, its file and line, and the evidence quoted. Every claim that was drawn from the documents is listed with what became of it. A block at the end records which model answered in each role, the settings, the cost and whether document content left the machine.

The exit code tells a script how the run went:

Code	Meaning
0	Nothing was found.
1	There is a problem in the merged document. The runbook merge of the opening story ended this way.
2	The run could not finish, or could not check everything. I call such a run *inconclusive*.
3	Milder than 1: the merged document passed every check, but the merge's records are wrong or incomplete. Example: a record says a sentence was replaced, and the sentence is still in the merge unchanged. Or the sources head the same section differently, and the merge kept one heading and wrote no record for the other.

When more than one applies, the run returns the most serious: 2 before 1, and 1 before 3. An inconclusive run is not a pass. A run that checked nine claims of twelve has not shown the merge clean, so any unfinished part makes the whole run inconclusive.

3 How the checks work

This section explains what happens between the merge and the report. The checks described here are the guardrails of Section 1, taken one at a time.

A merge can go wrong in three ways: it can drop something, change something, or invent something. A checker that is itself a language model can go wrong in the same three ways. So what can be decided by comparing strings is decided that way, and a model is asked only about meaning. That gives two families of checks. The *mechanical checks* are a program. A reader can verify any of their findings with a text editor. The *judged checks* rest on a model, the judge. The tool keeps the two apart in the report, and Section 5 leans on that split. Figure 3 shows the steps of a run in order.

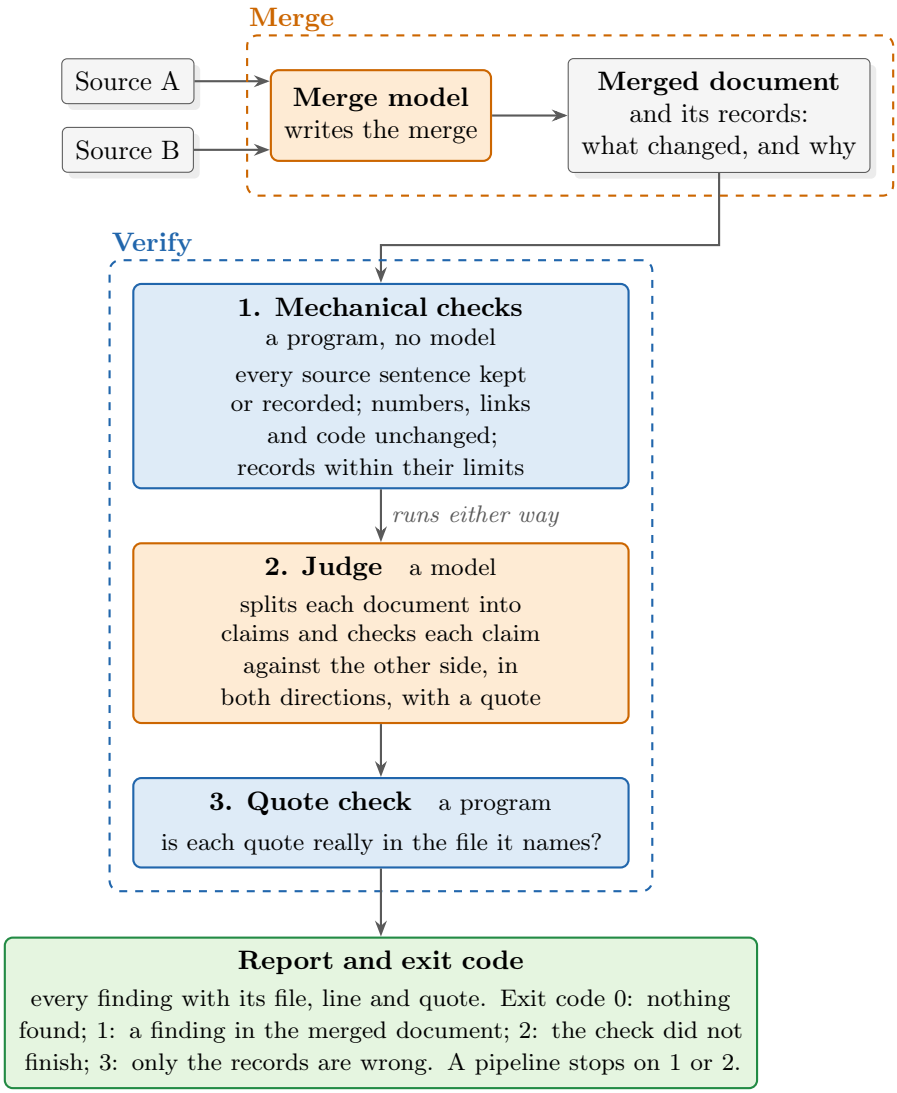


Figure 3: What happens in one run, in order: the merge, then its verification. `llossless merge` does both; `llossless verify` does only the second, on a merge made elsewhere. Blue boxes are a program, which compares text and cannot invent anything. Orange boxes are a model. A model writes the merge, and a model judges what only meaning can settle, but every quote it gives as evidence is checked by a program.

3.1 What a program can settle

Segments. A program first cuts every document into *segments*. A segment is a sentence, or one of the other units a document is built from: a heading, a list item, a table row, a block of code set off by fences. The field crew's sentence "Escalate E2 to the workshop after 2 failed attempts." is one segment. A segment also ends where a line of the file ends on a comma or another punctuation mark. So the support desk's sentence "If the field crew have already tried 3 times, raise it to the workshop instead." is two segments, cut after the comma, where its line ends. Sources and merge are cut the same way, so the program can look for a source segment in the merge without asking a model.

Protected values. Some parts of a segment must never change: numbers, version strings, inline code, file paths, links and URLs. I call them protected values. A fenced code block is protected byte for byte. A merge may rewrite the sentence around a command that is written as inline code, but not the command. It may not re-indent a fenced YAML block.

Records. The merge model returns the merged document and one record for every source segment it did not copy word for word. A record names the segment, says what happened to it, and gives a reason. There are six kinds:

Kind of record	What the merge says happened to the segment
reworded	The same content is in the merge in other words.
superseded	Another source's version of this content was used instead.
subsumed	The content is inside a broader statement of the merge.
duplicate	Another segment already says this.
combined	This segment and another were joined into a statement that neither makes alone.
dropped	The content is not in the merge, and nothing replaces it.

In the opening story the record for the support desk's sentence is of the kind *superseded*, with the reason "Threshold conflict resolved in favour of the base value." A record of the kinds reworded, superseded, subsumed or duplicate also names a *replacement*: the text in the merge that is said to carry the content now.

The rule that makes loss checkable. Whether a merge lost anything is a question no program can answer by itself. A sentence that is missing from the merge may have been reworded, folded into another sentence, or dropped. The records turn this into a question a program can answer. Every source segment must be in the merge word for word, or it must have a record. For a segment that is not there word for word, the program looks further, by similarity and by the run of its words (Appendix A). If it finds the segment in altered form, the segment counts as reworded, and a reworded segment with no record is a finding. If it does not find it, the segment is missing. A segment that is missing and has no record is a silent loss, and the mechanical checks report it. Figure 4 draws the whole rule.

The program also checks the records against the text, as far as a comparison of text can. The replacement that a record names must be in the merge. And a record must not describe a change that did not happen. A record may say that a segment was subsumed or superseded while the merge still carries that segment unchanged. No content is lost then, but the merge's account of its own work is false. Exit code 3 is for this case (Section 5.4), and for an *untaken heading*: a source heading that the merge did not take and wrote no record for, while its section is in the merge under another heading.

Two limits on records. A model could use records to excuse any loss, so two rules limit them. The first is the *loss budget*. A merge may declare segments dropped, with records of the kind dropped, but not more than 3% of the source segments it was given. That figure is the default, and the user can change it with the option `--loss-budget`. The second limits a replacement. One replacement may stand in for at most 3 missing segments. A single sentence that claims to carry many detailed ones has dropped them. Section 5.4 says where the second rule came from and what it still misses.

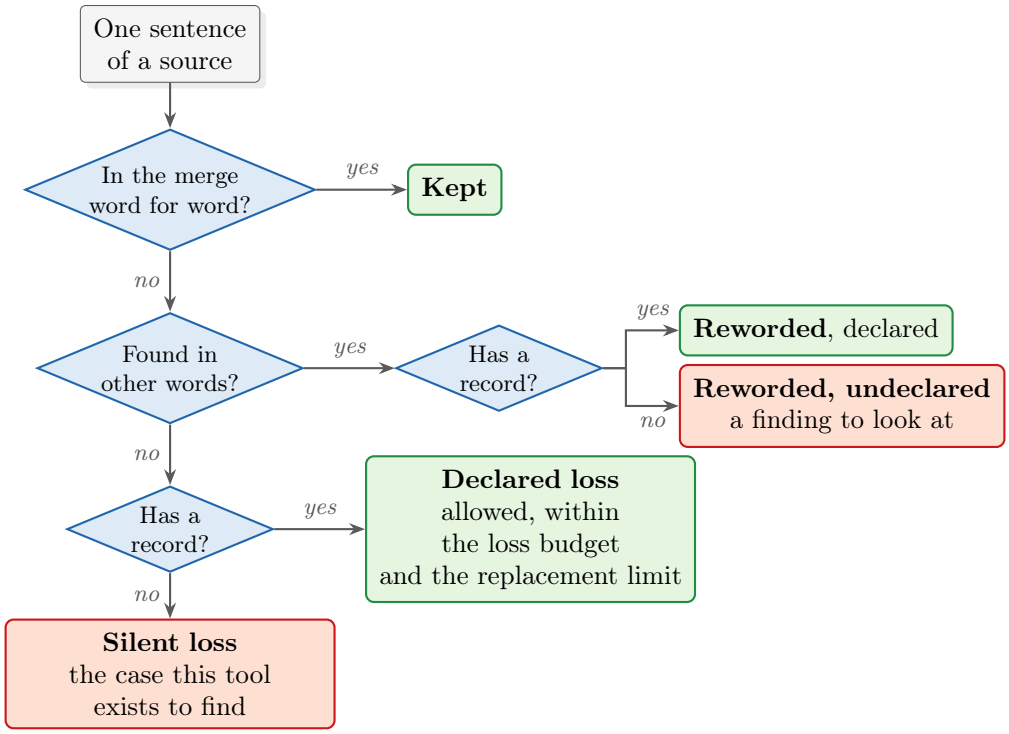


Figure 4: What becomes of one sentence of a source. Every question here is answered by a program. Green outcomes are allowed. Red ones are reported, and a silent loss is the case the tool exists to find. Because the program compares text and not meaning, it can call a sentence missing that the merge kept in very different words, so a silent loss is a place to look and not proof.

How the mechanical checks decide. There are 9 mechanical checks, built from four operations. A search looks for one text in another, character for character or word for word. Regular expressions cut documents into segments and find protected values. A similarity score with a fixed cut-off helps to tell reworded from missing. The rest is counting. The code is plain Python with its standard library, `re` for the patterns and `difflib` for the score. No model is asked, so the same input gives the same findings. Appendix A lists the checks one by one, says how a segment is looked for, and gives the patterns for protected values.

What the program reports on two real merges. Here are two merges of the runbooks from Section 1, taken from the benchmark. The program caught both, and it judged them differently.

Example 1: a near-duplicate dropped.

Field crew	Report E4 to the electrical contractor the same day.
Support desk	Raise an E4 to the electrical contractor the same day.
Reference merge	Keeps both sentences.
Merge	By `haiku-4.5-sub`: keeps the field crew's sentence, drops the support desk's, writes no record.
Program	`is reworded in the merge and no disposition record explains it`

The program looked for the support desk's sentence and found the field crew's, which is nearly the same. So it reports the sentence as reworded without a record. That is listed for a person to look at, but it is not a silent loss. Both readings of the audit found the content still in the merge.

Example 2: a number dropped.

Field crew	Escalate E2 to the workshop after 2 failed attempts.
Support desk	If the field crew have already tried 3 times, raise it to the workshop instead.
Reference merge	Keeps 2. Its record says that the support desk's sentence was superseded by the field crew's.
Merge	By `sonnet-5-api`: keeps 2, drops 3, and writes the record "Attempt-count disagreement resolved to the base's value of 2."
Program	`numeric '3' (times) does not survive into the merge unchanged`

The model made the reference merge's choice and declared it, but a number is a protected value. A record excuses a missing number only when the replacement it names is the wording of a source sentence (Appendix A). The reference merge's record names the field crew's sentence as written. This record names it joined to the one after it, which no source has in that form. So the program reports the 3 as missing.

What the mechanical checks cannot see. When a record names a replacement, the program checks that the replacement is in the merge, not that it says what the missing sentence said. The merge of Section 1 is an example. Its record names the field crew's "Escalate E2 to the workshop after 2 failed attempts." as the replacement for the support desk's sentence with 3. Unlike the record in the second example above, it names a sentence that stands in the merge exactly as a source wrote it, so the mechanical checks accept the record. That the two numbers disagree is a question of meaning, and it was the judge that reported it. Such questions go to the judge (Section 3.2). Section 5.4 shows how far a merge could push this.

3.2 What needs a judge

Whether "refund at the time of the call" survived in a sentence that says "refund immediately" is a question about meaning. For such questions the tool asks a model, the judge.

Claims. The judge first splits each document into *claims*, short statements of one fact each, with the line each comes from. In the literature these are called atomic claims. Segments and claims are different things. A segment is a piece of the text as it stands, cut by a program. A claim is a statement of what the text says, written by a model. One segment can hold several claims. The field crew's runbook has the sentence "E2 is a mechanical fault and E4 is an electrical one, and they are escalated differently." From it the judge drew three claims: "E2 is a mechanical fault.", "E4 is an electrical fault." and "E2 and E4 are escalated differently."

Both directions. The judge then compares each claim with the other side and gives a verdict. Every source claim is compared with the merge. Every merge claim is compared with the sources. Three verdicts are findings:

- A source claim that is missing from the merge was *dropped*.
- A merge claim that is missing from the sources was *invented*.
- A claim that the other side states differently is *contradicted*.

Checking both directions is the default. The tool calls it the verification depth `full`. A cheaper depth, `coverage`, compares only the source claims with the merge. It can find what was dropped, but it cannot find what was invented. Section 4.6 measures the difference.

A verdict must quote its evidence. A judge can be confidently wrong. So a verdict that rests on what the other side says must quote it. The quote must be *grounded*: the tool looks for the quoted text in the file the judge named, and it has to be there. This is one of the two findings about the count in the opening story:

> `source_b.md:15` says: If the field crew have already tried 3 times on an E2 fault, the fault is raised to the workshop instead.
>
> `merged.md` says: 'Escalate E2 to the workshop after 2 failed attempts.' (grounded)
>
> judged against: `merged.md`
>
> why this was read as a contradiction: The reference text sets the escalation threshold at 2 failed attempts, not 3.

These are the lines of the report, as it prints them. The first is a claim that the judge drew from the support desk's runbook, in the judge's own words, with its file and line. The second is the quote that the judge gave as evidence. "Grounded" means that the tool found that quote, word for word, in the merged document. The judge calls the document it checks against "the reference text", and here that is the merged document. A person can check the finding by opening both files at the places named. A quote that is in none of the files is reported as a transcription error.

When the judge did not finish. Two cases make a run inconclusive, with exit code 2. In the first, claims were graded and not one quote could be grounded, so nothing was measured. In the second, the judge failed to grade a claim even after a fixed number of retries. The claim is then listed as not graded. This happens when the judge's answer for that claim breaks the required format, for example by naming a claim it was not asked about, and the judge is told what is wrong and still gets it wrong on every retry.

3.3 Running against real endpoints

Two practical matters decide whether the checks can be relied on from day to day. One is whether a model's answer arrives in a form a program can read. The other is whether the tool's own tests can run without a model.

Answers a program can read. Every model answer is JSON that must match a schema, which is a formal description of the fields the answer must have. Endpoints differ in how firmly they can be held to a schema. The tool knows three ways and uses the strongest one the endpoint supports. In the strongest, the endpoint enforces the schema itself. In the second, the schema is the parameter list of a function the model must call. In the weakest, the schema is only described in the prompt. The report records which was used.

One lesson came from this. A schema with a single optional field made one vendor's strongest mode refuse the schema. The tool then fell back to the weakest way, and no error appeared anywhere. Since then no schema the tool sends has an optional field, and a test holds the merge schema to that at every fidelity level.

Tests without a model. The test suite needs no model. The model answers its figures depend on were recorded from real models and are replayed with no network. Each recorded answer is stored under a digest of the request that produced it. So when a prompt changes, the request changes, and the old recordings no longer match. The figures that depended on them are then reported as *unmeasured*, not as passes.

4 What it measured: a dated snapshot

This section reports what current models did when they merged the same documents under the tool's checks, on one date. The main question is which models lose content without saying so, and which merge best. Smaller measurements follow: which model fixes errors in its sources, which notices a bad merge, whether the route matters, what the cheaper check misses, and what the local models I tried first did.

A snapshot, not a leaderboard. Models changed faster than I could measure them (Section 6). So the figures describe one fixed set of models, measured in one session that began on 2026-09-27 and ended shortly after midnight UTC on 2026-09-28, with one fixed version of the tool.

Written down before the run. The benchmark was *registered*: its question, its models, its settings and its rules were written into a file in the repository, in the directory `arms/2026-09-27/lineup`, before the first model call. A later correction goes below the original text as a dated *amendment*. The file carries 6 amendments. The first and the second were written before the calls they govern. The third and the fourth are of another kind. I wrote both on 2026-10-12, after every call and with the results known, and Section 5.3 is about them. The fifth and the sixth, of 2026-10-15, correct the reference merges of the runbook pair and the loading-dock pair. Each had kept a value that a merge at the default level is told to choose away or remove, the support desk's count and the hours of a retired phone line, in a sentence of its own that is in neither source. Only the amended scoring's deviations on those pairs move, and the order does not.

4.1 What was run, and how it was scored

The models and the configurations. The benchmark covers models of the vendors Anthropic and OpenAI. Anthropic's are `claude-opus-5-5`, `claude-sonnet-5` and `claude-haiku-4-5-20251001`, which I call Opus, Sonnet and Haiku. Opus is the Opus 5.5 of the opening story, and the merge told there is from the configuration `opus-5.5-api`. OpenAI's are `gpt-6-sol` and `gpt-6-luna`, which I call the GPT models. All of them are hosted models. Opus, Sonnet and `gpt-6-sol` are *frontier models*. Haiku and `gpt-6-luna` are the small, cheaper model of each vendor. Take the list price of output from `gpt-6-luna`, the cheapest, as the unit. Output from Haiku then costs 10 times as much, from Sonnet 20 times, from `gpt-6-sol` 20 times, and from Opus 40 times.

The Anthropic models ran on the API route and on the subscription route, and the GPT models on the API route only. That gives 8 configurations. The name of a configuration joins the model and the route: `opus-5.5-api` is Opus through the API, and `opus-5.5-sub` is Opus through the subscription. In every run the configuration's model filled all the roles: it wrote the merge, and it was the judge of that merge. That is how I set the benchmark up, and it is not a limit of the tool (Section 2). Every call to the judge is a new request with a fresh context, so the judge sees the documents and the claims and nothing of the conversation in which the merge was written.

Another Anthropic model, Fable (`claude-fable-5-1`), ran through the subscription on a few documents only. I chose those documents after the results of Opus were in and before any call to Fable, as the second amendment records. Fable appears once, in Table 2. Gemini 3.8 Flash was to run beside the others and has no figure: its vendor's free tier refused every call, and I did not run it on a paid tier, for reasons of budget.

The test material. Every configuration ran the same files.

- *Merging.* The 9 pairs, merged at the default level `high`. A pair is the same procedure written down twice, as separate teams might write it for different readers. The bike-share runbooks are one pair. Others are about visitor badges, a payroll deadline and a freezer alarm. Every pair has a reference merge, with the records that belong to it. The files are in `tests/pairs/`.
- *Fixing errors.* The Voyager set and the seed-phrase set, whose sources carry planted errors, and the Mah-Jongg set, which carries none. These were merged at `open`, the level that lets a merge correct a source.
- *Detection.* The 13 test cases that I had registered for this purpose (Section 4.4).

How often each ran. Most pairs ran once on each configuration. The Voyager set and the seed-phrase set ran 3 times on every configuration, and so did the pairs `badge_access`, `trace_names` and `rate_limits`, except on `sonnet-5-api` and `haiku-4.5-api`. There the first amendment took those repeats out, to keep the cost of the Anthropic API under the cap I had set. I call these the repeated pairs. A model does not give the same answer twice, and the repeats show how far a figure moves from one run to the next. No answer was shared between runs, except that a run started again after a failure reused the answers its own earlier attempt had received.

The rule for ordering. The rule is not a combined score. First, a configuration that lost content with no record on the first run of any pair is disqualified. The audit decides that, not the program's count. Then the others are ordered by their deviations from the reference merges, fewest first. Configurations with the same number share a place, unless all of them are billed at list price. Then the lower cost decides. A weighted score would let a cheap model buy back lost facts with its price, which is the trade this tool exists to refuse. The rule encodes my requirement, as little loss as possible, and a user who wants the lowest price would order the table differently. Because most of it rests on a single run, I call the result an order and not a ranking.

In total the benchmark cost $34.29 at list price for the Anthropic API configurations and $7.26 for the OpenAI ones. The subscription configurations came to $49.47 API-equivalent, which is not a bill. The earlier measurements on rented GPUs (Section 4.7) cost several times as much.

What was not the same for everyone. The route differed. The judge graded 25 claims per call over the API and 100 through the command-line program. Only the OpenAI configurations sent a seed, the setting that makes a model's answers more repeatable. And the reasoning effort, a vendor setting for how much a model reasons before it answers, differed.

4.2 Who loses content, and who merges best

On the first runs the program counted a silent loss for 4 of the 8 configurations (Table 1). A count is a flag, so the audit read the segments: 44 on the first runs, each of them twice. They were the 25 that the registered scoring had counted as silent loss and the 19 that the program had passed as reworded although no record named them. In 6 of them it found content gone, wholly or in part, with no record. Those 6 are in the merges of 3 configurations, and the rule disqualifies them: the Haiku configurations, `haiku-4.5-sub` and `haiku-4.5-api`, and `gpt-6-luna-api`. Each is the small model of its vendor, and no configuration of a frontier model is among them. Each Haiku configuration lost content in more than one place. `gpt-6-luna-api` is disqualified for one sentence: it lost its opening condition, with no record, and both readings call the case borderline. I ruled that it disqualifies, and Section 5.3 quotes it. `sonnet-5-sub` is the one configuration with a counted

silent loss that is not disqualified. The audit read 9 of its segments and found every one of them in the merge.

Of the 5 configurations that are not disqualified, `gpt-6-sol-api` comes first under the rule. It has the fewest deviations of any configuration, 3.89 a pair, and no counted silent loss on a first run. `opus-5.5-sub` and `sonnet-5-sub` have 5.67 deviations a pair each and share the second place. Cost does not decide between them, because their costs are API-equivalents and not list prices. `opus-5.5-api` and `sonnet-5-api` come last, at 5.78 and 5.89. `sonnet-5-api` is also the slowest API configuration. Figure 5 sets the deviations against the cost of a merge.

The baselines. The rows at the foot of the table are texts made by a script from the sources of a pair and scored like a merge. *Sources concatenated* is both sources pasted one after the other. *Mechanical union* is one source followed by every paragraph of the other that the first does not already contain. *Base only* is the base source alone. The mechanical union loses nothing and has 7.78 deviations a pair. That is the figure a model has to beat to have done more than a script. Every configuration beats it: each has fewer deviations than the mechanical union on the pairs it finished. Concatenating the sources loses nothing either, which is why a benchmark that scores only whether the sources survived cannot tell a model from a script. Base only loses whatever only the other source says. The program counts 121 segments over the 9 pairs as silent loss, and 8 headings as untaken. Its deviation figure still looks ordinary, because a reference merge replaces most segments of the second source with a record for each and the deviation count does not look at records. That is why the order needs both measures.

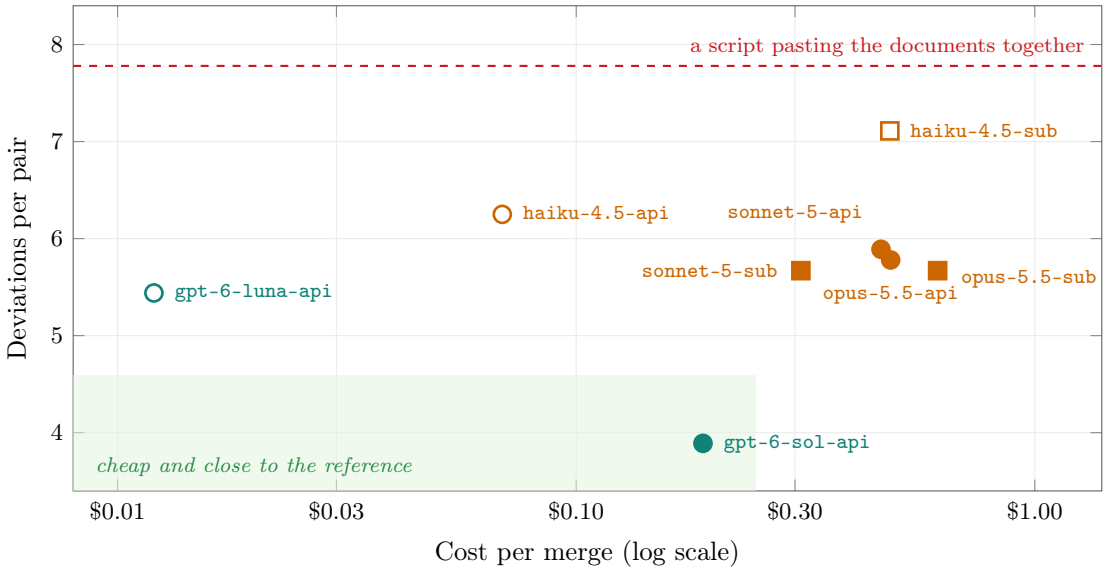


Figure 5: Price against merge quality on the 9 pairs: lower is closer to the reference merges. Orange is Anthropic, teal OpenAI. Circles are the vendor's API, squares its command-line program under a subscription, whose cost is the API-equivalent and not a bill. Hollow markers are disqualified for lost content. The dashed line is a script with no model; every configuration is below it.

What the registered scoring said. Under the scoring as registered, the program's count disqualified, and it disqualified 5 of the 8 configurations: the Haiku configurations, `gpt-6-luna-api`, `opus-5.5-api` and `sonnet-5-sub`. But the registered count was mostly not lost content: the reading

found content gone in 3 of the 25 segments it counted on first runs, and the count of 2 that disqualified `opus-5.5-api` was 2 untaken headings. The registered scoring also found that one model did no better than a script: `sonnet-5-api` had 8.11 deviations a pair against 7.78 for the mechanical union, where the amended scoring has 5.89 against 7.78. `gpt-6-sol-api` is first under both scorings. Every figure of the registered scoring is in `figures.md` in the benchmark's directory.

Configuration	Order	Pairs done	Silent loss	Untaken headings	Lost, by reading	Deviations /pair	Cost /merge	Seconds /merge
`gpt-6-sol-api`	1	9/9	0	0	–	3.89	$0.189	158.4
`opus-5.5-sub`	2=	9/9	0	0	–	5.67	$0.614	164.1
`sonnet-5-sub`	2=	9/9	2	0	0 of 9	5.67	$0.309	186.5
`opus-5.5-api`	3	9/9	0	2	0 of 2	5.78	$0.485	142.7
`sonnet-5-api`	4	9/9	0	0	–	5.89	$0.462	332.4
`gpt-6-luna-api`	disq.	9/9	2	0	1 of 4	5.44	$0.012	141.3
`haiku-4.5-sub`	disq.	9/9	3	0	3 of 8	7.11	$0.483	676.5
`haiku-4.5-api`	disq.	8/9	3	9	2 of 21	6.25	$0.069	69.1
sources concatenated	–	9/9	0	0	–	8.78	–	–
mechanical union	–	9/9	0	0	–	7.78	–	–
base only	–	9/9	121	8	–	7.56	–	–

Table 1: Merge quality on the 9 pairs: the first run of each pair, under the amended scoring. The rows in italics are baselines made by a script.

Order: place under the rule. `disq.` is disqualified by the audit, `=` a shared place. *Silent loss*: source segments that the program could not find and that no record explains, summed over the pairs. *Untaken headings*: source headings that the merge did not take, with no record, while their section is in the merge. *Lost, by reading*: of the segments that the audit read, those it found gone, wholly or in part. *Deviations/pair*: source segments that the merge treats differently from the reference merge (lost, kept where the reference left them out, or repeated). *Cost/merge*: list price for `-api` rows, API-equivalent for `-sub` rows, which is not a bill. *Seconds/merge*: the model's answering time over all roles. In every column after *Pairs done*, lower is better.

How firm the order is. The order rests on first runs, and the repeated pairs show how much of it is firm. The first place is firm. Over the 9 pairs `gpt-6-sol-api` has 16 fewer deviations than the configurations in second place, and no configuration in the order moved by more than 3 deviations between its runs of a repeated pair. So the lead is larger than any movement I measured. The places behind the first are not firm. The other 4 configurations lie within 2 deviations of each other over all 9 pairs, which is less than a single repeated pair moved between runs, and a second run could put them in another order.

4.3 Who fixes factual errors

This test asks a different question. Source documents can be wrong. At the level `open` a merge may correct a source from what the model knows, if it declares the correction (Section 2). So I planted errors in the sources of the Voyager set and of the seed-phrase set, and counted how many of them a merge fixed.

Each of these sets has a reference merge that carries the correct facts. Its sources say what the reference merge says, word for word, except where I planted an error. One of the errors: a Voyager source says that the names of some moons allude to Goethe, where the reference merge says Shakespeare. The Voyager set carries 44 planted errors and the seed-phrase set 14. The published scoring program builds the answer key itself. It compares the sources with the reference merge word

by word, and every difference is a planted error. An error counts as fixed when the merge has the wording of the reference merge and not the planted wording. A program decides that, not a judge.

The German Mah-Jongg set carries no planted error. It is the control, which means it shows what a model does when there is nothing to fix. A sentence of the merge that replaces words of its source there counts as a *false correction*.

A reminder from Section 2: a correction contradicts a source, so the tool reports it as a finding even when it is declared and right. This test does not count the tool's findings. It counts what the merged text says. Table 2 gives the counts.

Configuration	Order	Voyager errors fixed	Seed-phrase errors fixed	Fixed, both sets	Mah-Jongg false corrections
`opus-5.5-sub`	1=	30/32/35	13/13/13	45	1
`opus-5.5-api`	1=	28/29/32	13/13/13	42	0
`gpt-6-sol-api`	2=	22/23/29	10/11/12	34	2
`sonnet-5-api`	2=	20/21/23	12/12/12	33	0
`sonnet-5-sub`	2=	18/20/23	12/12/12	32	0
`gpt-6-luna-api`	3=	18/19/19	7/8/10	27	0
`haiku-4.5-api`	3=	6/11/18	8/8/9	19	1
`haiku-4.5-sub`	3=	2/6/17	6/8/9	14	failed
`fable-sub`	–	38	–	–	–

Table 2: Planted errors fixed, of 44 in the Voyager sources and 14 in the seed-phrase sources. *Order*: by the errors fixed on both sets, the medians added, then by fewer false corrections; = for a shared place, where a row's ranges overlap those of the row above on both sets. The order is for this test only. Each cell gives the lowest, the median and the highest count over 3 runs, separated by slashes. Higher is better. *Mah-Jongg false corrections*: sentences of the merge that replace words of a source that held no error. The Mah-Jongg control ran once. Lower is better. `failed`: the run ended as a model failure and produced no merge. The last row, Fable, ran Voyager only, once, and fixed more errors than any other run.

Opus fixes the most factual errors. Of the 8 configurations, the Opus ones fixed the most, on both routes (Table 2 and Figure 6). On the seed-phrase set both Opus configurations fixed 13 of 14 on every run, which no other configuration reached on any run. On Voyager I compare ranges, where a range runs from a configuration's lowest count to its highest. The subscription configuration's range lies above the range of every configuration of another model. The range of the Opus API configuration overlaps that of `gpt-6-sol-api`, whose best run fixed 29 against the Opus API configuration's worst of 28. `gpt-6-sol-api` also made the most false corrections on the control. On Voyager, Haiku fixed the fewest on both routes, with the widest range. Fable ran Voyager once, through the subscription, after Opus's results were in. It fixed 38, above both Opus ranges.

An error almost nobody fixed. The seed-phrase set is a test document adapted from BIP-39, the Bitcoin proposal that defines the recovery phrase of a wallet (`tests/handwritten/bip39/` in the repository). A planted error in it changes the proposal's licence from MIT to GPL. Of the 24 runs, 2 put MIT back. 8 kept GPL, 2 dropped the line, and 12 replaced it with the 2-clause BSD licence, which the proposal does not carry. All 12 declared the change and gave the proposal itself as their source. The report listed each one for a person to check, and every one of those runs ended with exit code 1. That is the tool working as intended: it cannot tell a wrong correction from a right one, so it stops the run and leaves the decision to a person, who would have found all 12 of them wrong. The benchmark ran at `open`, where a model corrects from memory and cannot look anything

up, so those citations came from memory. In an earlier, smaller test at `sourced`, where a model may search the web, 21 of 27 runs looked something up and 20 of those wrote MIT. None of the 6 that looked nothing up did. That test ran on an earlier version of the tool, so it is a pointer and not a measurement.

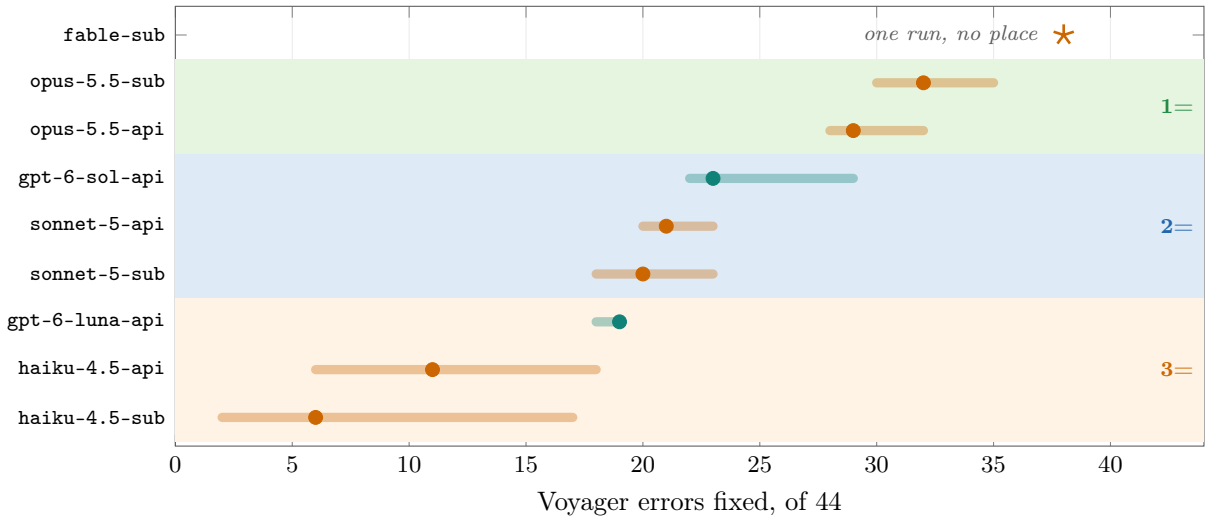


Figure 6: Voyager errors fixed, from each configuration's lowest run to its highest, with the median as a dot. Orange is Anthropic, teal OpenAI. Where the ranges of neighbouring rows overlap, a further run could swap them, so they share a place in Table 2: the shaded bands. Fable ran once, the star at the top: the best single run, but with no place.

No configuration is best at everything. A user wants a model that loses nothing, stays close to the reference merge, fixes errors and costs little. No configuration leads on all of these. `gpt-6-sol-api` merges closest, and it is in the middle on fixes. It is also the cheapest configuration that is not disqualified. `opus-5.5-sub` fixed the most errors, and it has no counted silent loss on any run. Its weak point is price: at $0.614 a merge, API-equivalent, it has the highest cost of any configuration. `gpt-6-sol-api` costs about a third of that.

Haiku is not worth using for merges. On paper Haiku, Anthropic's small model, is the obvious choice for a cheap pipeline: through the API it was the fastest configuration, at 69.1 seconds a merge, and one of the cheapest, at $0.069. The benchmark says otherwise, on either route. Both of its configurations are disqualified, and each lost content in more than one place (Table 1). `haiku-4.5-api` also left 9 headings untaken on the 8 pairs it finished, more than all the other configurations together. Both configurations fixed the fewest Voyager errors, and both model failures of the benchmark are theirs: runs that could not finish because the model's answer was unusable even after the tool's retries. And `haiku-4.5-sub` is the slowest configuration, at 676.5 seconds a merge.

The cheapest configuration is out as well, on much less. `gpt-6-luna-api` costs $0.012 a merge, the least of any configuration, and it is disqualified (Section 5.3). By its deviations, 5.44 a pair, it would stand second. On both sets with planted errors its median is below that of every configuration of a frontier model.

What the guardrails did. The runs of the benchmark were made by the tool as it was then, which looked for a segment by the registered rule. In 18 runs of the pairs, repeats included, the scoring program counted a silent loss, 65 segments in all. In all 18 of them the tool had already said so. Its own report of that run lists every one of those segments as missing with no record, and the run ended as failed. These merges read as fluently as the others. A pipeline that acts on the exit code would have published none of them.

The reading shows what such a flag is worth. Of the 65 flagged segments, repeats included, 11 had lost content, wholly or in part, and the other 54 were still in the merge. So most flags were false alarms, but every loss among them was in a run the tool had stopped. The amended rule lowers the cost of looking: it flags 45 of those segments, in 10 runs, and all 11 with a loss are among them. The tool had reported the other kind of segment as well: the 19 on first runs that it called reworded with no record are each on a line of their run's report, and each of those runs ended as failed. So every loss that the audit found, of either kind, was on a line of the tool's report, in a run that the tool had stopped.

4.4 Who notices a bad merge

Both Sonnet configurations got everything right: all 13 test cases, every planted defect, and no false alarm (Table 3). `gpt-6-luna-api` was worst, with 10 of 13 test cases right and 4 false alarms. `opus-5.5-sub` raised as many false alarms. No configuration missed a planted defect that could be scored for it.

The tool's judged checks are only as good as the judge. This test measures the models as judges. It uses the 13 registered test cases. Each test case is a small merge that was written for the test, with its sources and an answer key. The tool is run as `llossless verify`, so it makes no merge, and the configuration's model is the judge.

Configuration	Order	Test cases right	Planted defects found	False alarms: invented findings	False alarms: correct flagged	Test cases not scorable
`sonnet-5-api`	1=	13 of 13	7 of 7	0	0	0
`sonnet-5-sub`	1=	13 of 13	7 of 7	0	0	0
`haiku-4.5-sub`	2	12 of 13	7 of 7	1	1	0
`haiku-4.5-api`	3=	12 of 13	7 of 7	2	1	0
`opus-5.5-api`	3=	12 of 13	7 of 7	2	1	0
`gpt-6-sol-api`	4	11 of 13	6 of 6	1	0	1
`opus-5.5-sub`	5	11 of 13	7 of 7	4	3	0
`gpt-6-luna-api`	6	10 of 13	6 of 6	4	1	1

Table 3: The models as judges. *Order*: by test cases right, then by false alarms of both kinds together, `=` for a shared place. The order is for this test only, and each test case ran once. A test case is right when the run ends with the exit code its answer key expects. Higher is better for test cases right and defects found, lower for the other columns. An invented finding is a finding that the answer key does not account for. *Correct flagged* counts statements that the answer key marks as correct and that the judge reported as a problem. A test case is not scorable when the judge gave an answer that the answer key accepts as a fair reading, so that the planted defect counts neither as found nor as missed. There are 7 planted defects to find, because one test case has a defect in which each of a pair of values is credited to the wrong source, with an entry for each.

---

6 of the test cases have a planted defect. In one, a fact from a source is missing from the merge. In another, the merge states a fact that neither source states. In a third, the sources disagree on a

value and the merge keeps one of them without saying so in the text, as the runbook merge did. The other 7 test cases plant nothing. They are correct merges that a careless judge might flag, for example a merge that states the same facts in other words. The judge has to find every planted defect and must report nothing else. Table 3 gives the results.

Every configuration found every planted defect that could be scored for it (Table 3). The configurations differ in what they report where nothing is wrong. Sonnet, on both routes, is the only model with every exit code right, no invented finding and no correct statement flagged. `opus-5.5-sub` flagged the most correct statements.

One test case explains the column *Test cases not scorable*. In `numeric_drift` one source gives a rough figure, the other gives an exact one, and the merge keeps the exact one without saying so. Careful readers disagree on whether that contradicts the rough figure, and the answer key accepts either reading. The GPT configurations reported nothing on `numeric_drift`. That is an accepted answer, so the test case cannot say whether they would have found its planted defect, and the table counts it as not scorable for them.

Table 3 is about short test cases, each built around one kind of defect. It does not show that these models catch every defect in a real merge. The opening story is the counter-example. There the model that merged the runbooks also judged its merge, and not every model reported the conflict it had settled: the tool flagged it on 5 of the 7 merges that dropped the 3.

4.5 Whether the route matters

The Anthropic models ran on both routes, so the benchmark can ask whether the route changes the result. The rule, fixed on 2026-09-25 in the registration of an earlier comparison, is that the route makes a difference for a model only if the range of its results on a repeated pair on one route does not overlap the range on the other.

Opus is the only model with repeated pairs on both routes, and both of its routes ran at `medium` reasoning effort. The route makes no measurable difference for Opus. Its ranges overlap on every repeated pair and on Voyager. Table 1 does show differences between them. The subscription configuration has 51 deviations over the 9 pairs and the API configuration 52, which puts the API configuration in the next place. And the API configuration has 2 untaken headings where the subscription configuration has none. Both headings are from a single run, and its other runs of that pair have none. So neither difference says anything about the route. The reports also show that each subscription configuration was answered by the same model as its API configuration, so the route did not swap the model.

4.6 What the cheaper verification depth costs

The cheaper verification depth, `coverage`, checks only whether the sources survived in the merge (Section 3). This experiment measures what it saves and what it misses. I ran it on 2026-09-24, with one open-weight model, Qwen/Qwen3.8-27B-FP8, over the 13 registered detection test cases. Table 4 puts the depths side by side.

Coverage missed one planted defect that the full check found, in the test case `hallucination`, where the merge states a fact that no source states. A check that never reads the merge back against the sources cannot find that, and the registration predicted it. Of the defects coverage did find, 3 were found by a program and not by the judge: a separate check, not one of the 9 mechanical checks of Section 3, that finds a sentence crediting a source with something that source does not say. It runs at both depths. It also examined none of the 28 correct statements that only a reading of the merge can confirm. What it saves is in the last rows of the table.

	`full`	`coverage`
Planted defects found	7 of 7	6 of 7
Kind of defect out of reach	none	`hallucination`
Correct statements confirmed	116 of 118	90 of 118
Correct statements not examined	2	28
Correct statements wrongly flagged	0	0
Model calls	71	26
Seconds	2924.2	1432.8
Speed-up over `full`	–	2.04 times

Table 4: The verification depths `full` and `coverage` on the 13 registered detection test cases, with Qwen/Qwen3.8-27B-FP8 as the judge and each test case run once. `full` compares the sources with the merge and the merge with the sources. `coverage` compares only the sources with the merge. *Planted defects found*: higher is better. *Kind of defect out of reach*: a kind of defect that the depth has no step to find, whichever model is the judge. `hallucination` is the test case in which the merge states a fact that no source states. *Correct statements*: statements that the answer keys mark as correct. A good judge confirms them and flags none. *Model calls* and *Seconds* are sums over all test cases. Lower is better. A dash means that the row does not apply.

When coverage is enough. Coverage answers the question the tool was first built for, whether anything in the sources was lost. It finds a dropped fact and a contradicted one. I consider it a reasonable choice where the risk is loss and the merge had little freedom to add anything, as at the low fidelity levels, or as a cheap first pass over many documents. It is the wrong choice where the merge may state things from outside its sources, as at `open` and `sourced`, and wherever an invented statement would do more harm than a missing one. Coverage does not read the merge back, so an invention passes unseen.

What this does not show. This comparison ran once, with one model, Qwen/Qwen3.8-27B-FP8, on the 13 detection test cases. Each test case is a short document built to hold one planted defect or none. How the depths do on real documents, and with other models, is not measured. I had run both depths on hosted models before, but with mixed settings and without keeping the output, so those runs are not shown.

4.7 Local models: an approach I stopped

Before the benchmark, I ran the tool on open-weight models that I operated myself, on a local graphics card and on rented GPUs. I call these local models. I stopped. My own graphics card, an AMD card, was not up to the task: most of the models I wanted, with the context they need, did not fit on it. Rented GPUs, first as whole machines and then as a serverless service that bills by the minute, cost more than a subscription to a frontier model, for results that were worse. The *local-model study* ran in August and September on the models qwen3.8:27b, deepseek-r1:70b and gpt-oss:120b. A later comparison of a small and a larger model on 2026-09-16 is the one measurement in this paper that has no registration.

That comparison showed that the small model was no use as a judge. Over the runs each model finished, `qwen/qwen3-8b` drew 15 to 20 claims a run from the documents, and `Qwen/Qwen3.8-27B-FP8` drew 59 to 75 from the same documents. A judge that reads a fraction of the claims has looked at less, not found fewer problems.

One observation bears on every figure here. I ran gpt-oss:120b at temperature zero, with a fixed seed and the store of earlier answers off, the settings meant to make a model repeat itself. In one set

of 5 runs, made one after the other, it returned 2 distinct merges: the first differed from the rest. In a second set of the same size every merge was identical. The same output is not guaranteed, so a repeated call is a second sample and not a check.

So for this job I can recommend hosted models only, and not every hosted model either (Section 4.2). Whether a local model would match the stronger hosted configurations, this benchmark cannot say.

5 How the figures are kept honest

LLossless rests on one idea: a model's account of its own work is not evidence, so the check has to come from outside. The same holds for the tool itself, because agents wrote its code, its tests and this paper.

5.1 Figures and registrations

No measured figure in a result section of this paper is typed. Each is a name whose value a program generates from a file of results kept in the repository. The published file `paper/generated/numbers.json` lists every name with its value and its source. The build stops on a typed digit, on a name with no value, and on a sentence that compares figures when the figures no longer bear it out. That proves less than it seems, because a generated value only proves that a string came out of a file, and a figure can still count something other than its name says (Section 5.3). The benchmark's programs, documents and outputs are published, and anyone with API keys or a subscription can run it again. The result is a new sample, not the same figures.

Every experiment behind this paper was registered before its first model call, except one: the comparison of a small and a larger local model (Section 4.7). The benchmark has 4 amendments made after the results were known, and Section 5.3 is about them. The program that ran and scored the local-model study is strict. It refuses rather than warns, in ten places. It writes no score, for example, when the properties it can evaluate are not exactly the registered ones, or when the model being scored is also its judge.

5.2 Checks that passed while their claim was false

The most expensive failure in this project was a check that reported success while the thing it claimed did not hold. I count 20 of them. Most of the tool's mechanical checks were called by nothing except their own tests: on a real pair the report showed a single finding, and when I ran the uncalled checks by hand, they found several more. A detector for words copied from the merge prompt's worked example was never called by the shipped tool. A scanner that lists the files a program depends on could not see a dependency that came in through an import, its own among them.

Everything looked green until I entered data from outside, data the AI did not know, and suddenly a lot of it collapsed. The lesson is not to rely on the tests that the coding agent wrote itself, and to test with data the AI does not know while the app is being built. Since then a check has to be seen to run, a detector has to be seen to fire, and an expected value is derived and not written down. The mechanical checks are a named step of the command, and the report prints "Not checked" where that step did not run.

5.3 A figure that passed every check and miscounted

The same happened to the benchmark's headline figure. I asked how the mechanical checks actually work, so that this paper could explain them. Writing that explanation, the coding agent found that the headline was wrong. It said that 5 of the 8 configurations had lost content silently. What it counted was segments that a program could not find, and most of them were still in the merge: a sentence that the merge had joined to its neighbour with a semicolon, a heading that the merge did not take because it kept the other source's heading for the same section. The figure had been green from 2026-09-28, when the benchmark ended, to 2026-10-12. It was generated from the stored runs, and its sentence was held to it on every build. Nobody had read what it counted.

So the rule was repaired and the benchmark was scored again. An AI model then read the segments that a merge had changed or dropped without a record. By that reading 3 of the 8 configurations lost content, not 5. The repair and my ruling below are amendments that I made after the results were known, and so are later corrections to the reference merges of the runbook and loading-dock pairs (Section 4). Both scorings are published.

This rests on less than I would like. A model did the reading, a Claude model at that, and no person. And one of the 3 is out on a single borderline sentence, by my ruling. Its source says: "When a requested item arrives, print a hold slip and fold it into the item so the name is visible from the spine." In the merge by `gpt-6-luna-api` the words "When a requested item arrives" are gone, and the sentence stands under the heading "Placing an item on the shelf", which implies the occasion. A reader who takes the heading as enough finds no loss, and then `gpt-6-luna-api` lost nothing. The audit page in the repository, `arms/2026-09-27/lineup/audit/`, has every case with its quotations, and a count of what was not read.

5.4 A merge could declare its way past every check

A record switches the loss check off for its segment, and the check on a record asks only whether its replacement is in the merge (Section 3). So a merge could declare most of its sources away against one sentence. A test in the published suite shows it with plain Python and no model:

```
tests/test_reconcile.py
test_b5_declaring_your_way_past_the_checks_still_works
```

Its sources are release notes with 8 segments and 341 characters. The merge has 104 characters: a title and the single sentence "This release changes scheduling, queues, auditing, replicas, archiving and compaction." The 6 segments that are gone each have a record that names this sentence as their replacement. 75% of the source segments were gone, and with them 88% of the sources' characters. Every record was accepted, and every one of the mechanical checks passed.

The rule that now applies counts and does not read: one replacement may stand in for at most 3 missing segments. When I set it, no replacement in the reference merges of the 9 public pairs stood in for more than 3 missing segments, and I put the limit at that maximum, with no margin. With the rule in place the demonstration now fails the run. At every one of the 6 fidelity levels it returns 1 finding, because one replacement is declared to stand in for 6 missing segments.

The hole is narrowed, not closed. A merge can point each missing segment at a different part of the same surviving sentence. The variant with 6 records that name 6 different parts of the one sentence returns 0 findings at every level. A record that is coherent and false cannot be told from a true one without reading the documents, and reading is the judge's job. For a user this means that a clean result from the mechanical checks does not rule out a merge that summarised its sources away.

The opposite lie. A merge can also write a record that says a segment was subsumed or superseded and still carry that segment unchanged. In one recorded merge of a public pair by a small open-weight model, 15 of the merge's 30 records were false in exactly that way. No content is lost, but the record sends a reviewer looking for a change that is not there. Whether a segment is still in the merge unchanged is a comparison of text, so the check needs no threshold.

5.5 An answer key that contradicted itself

The detection test scores a judge against answer keys (Section 4.4), so a wrong key scores every model wrongly. That happened in the local-model study. A control test case planted no defect, yet its key said that a correct run should end as a failure, so the models that rightly found nothing were scored as having missed a defect. The expected exit code is now computed from the key's entries, and the scoring gained a third outcome beside pass and fail, *unmeasured*.

6 What building it taught

Each paragraph is one lesson from building the tool: what happened, and what I changed. The most costly one, checks that passed without running, is in Section 5.2.

The models moved faster than I could measure. After a change to the merge prompt, every merge I sent to `claude-opus-5` through its API was refused. The vendor's name for the reason was `reasoning_extraction`. Going back through the versions of the prompt, I found the one paragraph that caused it, a note that the sources are shown one segment per line. Without it the request was answered, and other models of the same vendor answered it with it. I dropped the model on 2026-09-25 for its successor, `claude-opus-5-5`. The requests of that search are published in `arms/2026-09-25/vendor`.

The program around a model moved too. After an update of the vendor's command-line program, one model stopped looking facts up at the level `sourced`, and only the report, not the exit code, showed it. A `sourced` merge that retrieved nothing now ends as inconclusive. My answer to both was to keep the tool and the measurement apart: the tool has versions and is tested on its own, and a benchmark is made with one fixed set of models and carries its date.

A nested command inherits its user's configuration. The subscription route runs Claude Code as a program inside the tool. Started plainly, it loaded my own configuration, including my instruction file `CLAUDE.md`, which is written for coding sessions and could shape a merge. The route now starts the program in its safe mode with only the tools the task needs, and the report records both. The benchmark of Section 4 ran in safe mode throughout. My earlier subscription figures were made with my configuration loaded and should not be compared with it.

Answer keys come from a diff, not from reading. The Voyager set was first scored against a list of errors that was typed by reading each source beside its reference. A word-by-word comparison of the same files found many more. Reading had found the loud errors and missed the quiet ones, a swapped word or a decimal comma. Those are exactly the errors a model carries through unnoticed. The answer keys are now built by the published scoring program and were reviewed error by error.

A claim cannot be graded by the words it is made of. I wanted a program, with no model, to decide questions of meaning: whether a merge really merged its sources or pasted them together,

whether a replacement carries what the missing sentence said, whether a title the merge wrote agrees with the sources. Each attempt was a measure of the text alone with a cut-off: similarity, the proportion of lengths, the count of missing segments, the words a title shares with the sources. On the test documents the values for correct and for defective merges overlapped, so no cut-off separates them:

- *A merge that says everything twice.* I compared the sentences of a merge with each other. The most similar pair of sentences in a document that is known to be correct was more similar than any pair in the merges that really stated both sources' facts twice. A cut-off that catches those merges accuses the correct document first.
- *A merge that only pastes.* Similarity to the pasted sources fired on a nearly complete copy. It missed a merge that had pasted both sources and dropped their titles and summary lines.
- *The length of the merge.* The proportion of lengths was higher for a merge with no repeated fact than for one with several, so it does not measure repetition.
- *A replacement that does not carry its segment.* Similarity between a missing segment and its replacement flags 32.7% of the correct records in the reference merges, and counting every missing segment against the loss budget puts a correct reference merge over the budget (Section 5.4).
- *A title.* The rule was that every substantial word of a rewritten title must occur in the sources. The example in the test suite: a source says "a quad feels more stable at walking pace", and the title is "A motorbike is more stable than a quad". Every word of that title is in the sources, so it passes the rule, and it turns the statement round.

A measure of the surface answers a different question, and its answer is worse than none, because it reads like a verification. So these questions are left to the judge. The fact stated twice, for example, is now found among the claims that the judge draws from the merge. The one rule of this kind that survived does not read at all. It counts records (Section 5.4), and it narrows its hole without closing it.

7 Limitations

This section lists what the tool does not do and what the figures do not show. Each paragraph is one limit.

It checks facts, not form. A merge that reads badly passes. Rewording within what the fidelity level allows is not reported.

Notation is the exception, and only in part. In a merge the tool made, where a source sentence is kept, the mechanical checks compare its numbers character by character. There, "512" rewritten as "0x200" is a finding, and so is "512" rewritten as "approximately 500". The judged checks decide about meaning and have no rule about notation. Under `llossless verify`, which checks a merge made elsewhere, only the judged checks look at such a change. A judge that reads both notations as the same number reports nothing.

Source code is outside what the tool protects, except inside fenced blocks of a document. In one test I gave the tool the same small program written once in C and once in C++. The programs were plain files, not fenced blocks inside a document. The merge kept the C version, dropped the C++ version and flattened the indentation. The run still ended clean, with every source claim accounted for, because both programs make the same claims about what the program does. What was lost was form, and the tool does not check form.

Loss can be declared away or misfiled. A merge can still declare most of its sources away against one surviving sentence and pass every mechanical check (Section 5.4). Catching that is left to the judged checks, and I have not measured how often they do.

A second weakness is in how a missing segment is classified. When the merge does not carry a source segment word for word, the program looks for a merge segment that resembles it. If one resembles it closely enough, the source segment is called reworded. Nothing requires that the resembling segment is really its counterpart. So when the sources hold two nearly identical sentences and the merge deletes one, the other is still there to be compared with. The deletion is then reported as a rewording, which flatters the merge. A sentence that was cut short is flattered in the same way: of 19 reworded segments with no record on the benchmark's first runs, a reading found a loss in 3 (Section 5.3).

The misfiling also runs the other way: a sentence that the merge states in other words is counted as a silent loss. Of the 10 segments that the amended scoring counts on the first runs of the benchmark, the reading found 7 present in the merge. So the silent-loss column of Table 1 still counts more than was lost.

The disqualification rests on a model's reading. An AI model decided by reading which configurations lost content, not a person, and one disqualification rests on my ruling on a borderline case. Section 5.3 gives the figures, what was not read, and the other limits.

A declared choice can hide a conflict from the loss score. A declared segment is not a silent loss, and a declaration can settle a disagreement that the merged text never shows. In the opening story the merge declared the dropped count, so the score recorded 0 silent losses for that run while the report flagged the contradiction and exited 1. A reader who looks only at the silent-loss column of Table 1 does not see such a case.

The judged half depends on its judge. The figures of the judged checks come from a model, and a different judge would give different ones. In the benchmark every configuration was judged by its own model. The judged half is not neutral between vendors and should not be read as if it were.

Drawing claims from a document does not always repeat. On the hosted configurations of the benchmark, the number of claims taken from an unchanged source moved between runs. On the local model behind the tool's recorded test answers, every first answer repeated.

Claims also get coarser as documents get longer. A claim that bundles several facts can be scored as covered by a merge that dropped one of them. How often that hides a loss has not been measured.

A small test set, built for properties. The test material is 9 document pairs with reference merges, 16 small test cases with answer keys, of which 13 form the registered detection test, and 19 further document sets. It was built so that each piece carries a specific property, such as a conflict that cannot be resolved by choosing. It was not built to be representative. No claim here is about documents in general.

The coding agents that wrote the tool also wrote the pairs, their reference merges and the answer keys of the test cases. So the yardstick can share a blind spot with the tool. The agents were Claude models, and Claude models are among those ranked against these merges. I reviewed the material, but nobody outside the project has.

The documents are also short. The longest pair of the benchmark, `rate_limits`, has 1,809 words in its sources, which the tool cuts into 104 segments. The largest document set that was merged is

`mahjongg`, the Mah-Jongg set, with 3,602 words in 216 segments. Nothing longer has been measured. So this paper cannot say what happens with documents of many pages.

Nearly every measurement covers two sources, although up to 12 are accepted. The exception is the Mah-Jongg set, which has 3 sources. No measurement covers more.

One snapshot, hosted models only. Most pairs ran once, and a second run is a fresh sample: no configuration sent a temperature, and only the OpenAI configurations sent a seed. So the figures support statements of the form "on these pairs, on this date, the program could not find so many segments in this configuration's merges, a reading found content gone in so many, and a merge cost so much". They do not support a ranking where ranges overlap or a ranking from a single run, and they do not support any combined score. No local model is in the benchmark. One measured document set is German: the Mah-Jongg set, whose sources also carry Chinese and Japanese characters. Nothing else is measured outside English.

One tool's opinion of a merge. The mechanical checks define what counts as a defect. They always give the same answer for the same input. That makes them reproducible, but it does not make them right: a segment they count as a silent loss is one a person might call an acceptable shortening.

The tool also merges whatever it is given. It does not decide which documents belong together.

And the web interface is for people who trust each other: a member without a key of their own runs on the operator's key.

8 Related work

This section places LLossless beside earlier work on checking what a model wrote. In short: I did not find a method that gives a verdict on a merge without resting that verdict on a model.

Atomic claims and faithfulness. FActScore [10] breaks generated text into atomic facts, which are single statements of one fact, and scores each against a knowledge source. RAGAS [2] scores how faithful an answer is to the passages it was generated from, with a model as judge. LLossless borrows the splitting into claims and adds the direction those tools do not need. They ask whether what was written is supported. An audit of a merge must also ask whether what was supported got written. A question-answering system is not expected to reproduce its context. A merge is.

Multi-document summarisation. Multi-News [3] and WikiSum [8] ask for one text that stands in for several, and score it by its overlap with a reference written by a person. Dropping a claim that the reference also dropped costs nothing there. FactCC [5] and QAGS [11] detect inconsistency between a summary and its source, and SummEval [4] re-evaluates the metrics themselves. A summary is supposed to omit, so a consistency metric does not count omission as a defect. A merge is not supposed to omit. So LLossless scores a merge against its sources, and keeps a reference merge only to measure deviation in its benchmark.

Hallucination and judges. HaluEval [6], TruthfulQA [7] and SelfCheckGPT [9] measure what a model states that is not true. For a merge, the check of merge claims against the sources covers that. The other half is not covered by any metric that is scored over what is present, because a claim that is absent is invisible to it.

Zheng et al. [13] studied how well a model serves as a judge and reported its biases, among them the position of an answer in the list it is shown. Wang et al. [12] show such a judge changing its ranking when the same candidates are shown in a different order. A metric that changes with the order of its input can inform a disagreement. It cannot settle one. That is my argument for keeping the mechanical checks apart from the judged checks (Sections 3 and 5).

Atil et al. [1] report that settings meant to make a model repeat itself, temperature zero included, do not guarantee the same output twice. I saw the same on one open-weight model (Section 4.7): a repeated call is a second sample, not a confirmation.

The gap. Leaving aside overlap with a reference, the metrics above rest on a trained model or a judge model, so a disagreement about a score is an argument about a model. The mechanical checks in LLossless are code. That is a deliberate trade: they measure less, and what they measure they settle.

9 How this was produced, and where to find it

How it was built. The idea is mine, and it comes from a real problem: merging documents that say the same thing with a model, and not being able to tell what the merge left out. I set the scope, wrote the plans and ruled on the design decisions. AI coding agents wrote the implementation, the test suite and the prose of this paper. I read what they reported and the decisions they recorded, not every line they wrote. The errors are mine.

It took about seven months, from a first proof of concept in March 2026. The subscription that ran the agents has usage limits, and at times the work stopped for days until they reset. Testing on local models took many hours for little progress (Section 4.7). And the scope grew. The first plan was the cheaper check alone, which asks only whether the sources survived in the merge (Section 4.6). That was not enough, so the full check, which also reads the merge back against the sources, was built on top of it, with more checks around it. Writing the test material by hand, the documents with planted errors and the reference merges, took a long time as well.

The disclosure is not a reason The disclosure is not a reason to discount the figures: the mechanical checks and the scoring program are code that anyone can run again. The exception is the disqualification, which rests on a reading by a model, and its cases are published (Section 5.3).

Where to find it. The repository is `https://github.com/toby-sutor/LLossless`. It holds the tool, the web interface, the programs that ran and scored the benchmark, the prompts, the test documents, the sources of this paper with the files of results behind its figures, and the published part of the test suite with recorded model answers. My working notes are withheld. For any figure, `paper/generated/numbers.json` names the file it came from and the place in that file (Section 5).

Which version this paper describes. The published repository starts a fresh history, so the title page names a digest of the published files instead of a commit: a SHA-256 over the path and the SHA-256 of every file, leaving out `paper/generated/`. `paper/README.md` gives the command that prints it in a fresh clone. The title page shows its first twelve characters.

The tool was renamed on 2026-09-26. Every report still carries the field `claimcheck_commit`, which keeps the earlier name and names the version of the tool that produced the report.

Reference: licences and credits. `NOTICE` in the repository is the authority and lists each exception. In summary:

- *This paper.* The text is under Creative Commons Attribution 4.0, as `paper/LICENSE` states: it may be shared and adapted with credit.
- *The code.* The code is under the Elastic License 2.0, as the repository's `LICENSE` file states, with the copyright holder named in `NOTICE`. It may be used, modified and redistributed, but may not be offered to third parties as a hosted or managed service.
- *The data.* The document pairs in `tests/pairs/`, the document sets in `tests/handwritten/`, the run outputs in `arms/`, the annotated pages in `annotated/` and the graded results in `paper/records/` are under Creative Commons Attribution 4.0, as a `LICENSE` file in each of those directories states, with one exception.
- *The exception: sets adapted from Wikipedia.* Four sets in `tests/handwritten/` are adapted from Wikipedia articles: `gold_de_en` from "Gold" in the German and the English Wikipedia, `mahjongg` from "Mah-Jongg" in the German Wikipedia, and `sepia` and `treecreeper` from "Cuttlefish" and "Eurasian treecreeper" in the English Wikipedia. Those four sets, and the benchmark run directories under `arms/` that merged `mahjongg`, are under Creative Commons Attribution-ShareAlike 4.0. Anything built from them must credit the Wikipedia contributors and carry the same licence.
- *The Voyager sources.* The `voyager` sources are adapted from a NASA page. NASA says its content is generally not subject to copyright in the United States, and it does not endorse this project.

References

[1] Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Non-Determinism of "Deterministic" LLM Settings, 2024. arXiv:2408.04667.

[2] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated Evaluation of Retrieval Augmented Generation, 2023. arXiv:2309.15217.

[3] Alexander R. Fabbri, Irene Li, Tianwei She, Suyi Li, and Dragomir R. Radev. Multi-News: a Large-Scale Multi-Document Summarization Dataset and Abstractive Hierarchical Model, 2019. arXiv:1906.01749.

[4] Alexander R. Fabbri, Wojciech Kryściński, Bryan McCann, Caiming Xiong, Richard Socher, and Dragomir Radev. SummEval: Re-evaluating Summarization Evaluation, 2020. arXiv:2007.12626.

[5] Wojciech Kryściński, Bryan McCann, Caiming Xiong, and Richard Socher. Evaluating the Factual Consistency of Abstractive Text Summarization, 2019. arXiv:1910.12840.

[6] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models, 2023. arXiv:2305.11747.

[7] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring How Models Mimic Human Falsehoods, 2021. arXiv:2109.07958.

[8] Peter J. Liu, Mohammad Saleh, Etienne Pot, Ben Goodrich, Ryan Sepassi, Lukasz Kaiser, and Noam Shazeer. Generating Wikipedia by Summarizing Long Sequences, 2018. arXiv:1801.10198.

[9] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models, 2023. arXiv:2303.08896.

[10] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation, 2023. arXiv:2305.14251.

[11] Alex Wang, Kyunghyun Cho, and Mike Lewis. Asking and Answering Questions to Evaluate the Factual Consistency of Summaries, 2020. arXiv:2004.04228.

[12] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. Large Language Models are not Fair Evaluators, 2023. arXiv:2305.17926.

[13] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena, 2023. arXiv:2306.05685.

A The mechanical checks in detail

The checks. Before a search, a run of spaces and line breaks becomes one space on both sides, and the Markdown marks at the start of a line are taken off. Punctuation, digits and word order have to match. The checks follow in the order of the report, each with the heading of its findings.

- *Every source segment is in the merge word for word, or has a record.* A search for the segment's text. A segment that is not found and has no record is reported under one of three headings: "Reworded and undeclared" if the program found it in altered form, "Absent and undeclared" if it did not, which is the silent loss, and "Heading dropped in silence" for a heading that the merge did not take while its section is still there. The paragraphs after this list say how the program tells them apart. A segment that is found although a record says it was subsumed or superseded is reported as "Declared gone, still here".
- *Every record names a real segment.* A lookup among the source segments. A record for a segment that no source has is an "Invented segment".
- *Every replacement is in the merge.* A search for the replacement's text. If it is not there, the finding is "Unresolved replacement".
- *Every record is of a kind its fidelity level allows.* A lookup in a table of levels and kinds. A kind the level does not allow is "Not permitted here".
- *Every protected value is in the merge unchanged.* Patterns find the values in a source segment, and a search looks for each of them in the merge. A value that is not found is a "Verbatim violation".
- *The records keep within the two limits.* The program counts declared drops against the loss budget of 3%, and missing segments per replacement against the limit of 3. Either limit exceeded is "Over budget".
- *The merge has a title.* It needs one if a source has one, and under the stricter title settings it must be a source's title, character for character. Otherwise the finding is "Title not from a source".
- *A source title that was not taken has a record.* The record must say superseded, and its replacement must contain the merge's title. Without such a record the finding is "Title dropped in silence".
- *The merge says nothing twice.* Its segments are compared with each other, titles and headings left out. Two segments that are equal word for word are reported as "Stated twice".

How a segment is looked for. A source segment that is not in the merge word for word is looked for in two further ways. The first is the similarity score. It is the ratio from `difflib`: twice the number of matching characters, divided by the length of both texts together. The segment is scored against every segment of the merge, and the best score counts. At or above the cut-off of 0.68 the segment counts as reworded. The score reaches the cut-off when a little more than half of a sentence is left, so a sentence that was cut short can pass as reworded. It is still reported, with the difference printed, but not as a silent loss (Section 7).

The second is for a sentence, a list item or a table row that the score left below the cut-off. The program asks whether the words of the segment still stand in the merge in their order, as one unbroken run, with punctuation and capitals ignored. That finds a sentence which the merge kept and joined to its neighbour: the word-for-word search misses it because its full stop is gone, and the score misses it because the joined sentence is much longer. The run may also reach across two neighbouring segments of the merge, for a sentence that was split in two or run together with the next. There a short list of connecting words may differ: the articles, "and", the forms of "to be", and pronouns such as "it" and "which". No other word may be missing or added, and every number, link and code value of the segment must be there. A segment that is found in either way is reworded. One that is found in neither is missing.

A heading is looked for only among the headings of the merge. A source heading that is not there and has no record is a silent loss if the whole section under it is missing too, or if it carries a number, a link or a code value that the merge does not have. Otherwise its section is in the merge under another heading, and only the label was passed over. The program reports that as "Heading dropped in silence". I call it an *untaken heading*. It is a finding about the merge's records and not about its content, so the run ends with exit code 3.

The patterns for protected values. The patterns find inline code, links, URLs, file paths, version strings and numbers. A number is a run of digits with what is attached to it, such as `30s` or `100%`, and a number written as a word is not found. Each value is searched for whole, so `512` is not found inside `5120`. At the default level a missing value is excused in two cases. Its segment has a record of the kind dropped, which the loss budget counts. Or it has a record of the kind superseded whose replacement is in the merge and is the wording of another source segment. The second case is for sources that write one number in two ways, such as "two billion" and "2,000,000,000".

B Words used in this paper

These are the paper's own terms, with the meaning each has here. Most are explained again where they are first needed.

source One of the documents to be merged.

merge The one document made from the sources, and the act of making it.

pair Two sources on the same subject that are to be merged. The benchmark's pairs each come with a reference merge.

segment One piece of a document: a sentence, a heading, a list item, a table row or a block of code. A program cuts every document into segments.

record The merge model's note about one source segment that it did not copy word for word. It says what became of the segment, and why.

declare To state a change in a record. A *declared loss* is content that the merge left out and wrote a record for.

silent loss A source segment that the program could not find in the merge and that no record explains. It is a flag for a person to look at, and not proof that content is gone.

untaken heading A source heading that the merge did not take and wrote no record for, while the section under it is in the merge. It is counted apart from silent loss.

reworded Said by the program of a source segment that is not in the merge word for word but closely resembles a segment of it. It is not counted as a silent loss.

audit The reading, against its merge, of every source segment on the first run of a pair that the merge does not carry word for word and has no record for. An AI model read each segment twice, in separate readings. The readings gave the same verdict on 42 of 44 segments, and where they differed, the first reading counts. *Audited loss* is content that this reading found gone, wholly or in part.

claim One statement of one fact, drawn from a document by a model, with the line it comes from.

judge The model that draws the claims from the documents and decides, claim by claim, whether the other side supports it.

mechanical check A check done by a program that compares text and asks no model.

judged check A check that rests on the judge's decision.

guardrails The checks around the model, taken together: the mechanical checks, the judged checks with their quoted evidence, and the exit code that stops a pipeline.

grounded Said of a quote that the judge gives as evidence: the tool looked for the quoted text in the file the judge named, and found it there.

verification depth The setting that says how much the judge checks. `full` compares the sources with the merge and the merge with the sources. `coverage` is cheaper and does only the first.

finding An entry in the report that names a problem, with the file and the line.

fidelity level The setting that says how much a merge may edit its sources (Section 2).

reference merge A correct merge of a pair or a document set, written in advance, to compare a model's merge with.

deviation One source segment that a merge treats differently from the reference merge: lost, kept where the reference left it out, or repeated.

configuration One model reached by one route.

route The way a model is reached. On the *API route* the tool calls the vendor's API, and every call is billed by the token. On the *subscription route* it runs the vendor's command-line program under a subscription: a fixed fee that buys a budget of usage, with no price per call.

frontier model One of a vendor's large, most capable models, as opposed to its smaller and cheaper ones.

run One execution of the tool on one set of documents.

test case A small merge written for testing, with a planted defect or none, and an answer key.

planted Put there on purpose. A *planted error* is a false fact that I put into a source. A *planted defect* is a flaw built into the merge of a test case.

registration A file that states the question and the settings of an experiment, written before its first run. A later correction to it is an *amendment*.

registered scoring, amended scoring The benchmark's results under the scoring rule as it was registered, and under the rule as I amended it after the results (Section 4).

End of document